Appello TAP del 19/02/2026 - Turno 1

Avete a disposizione **2 ore** (in questo turno non ci sono persone con DSA). La prova è composta da:

- **Quiz** su AulaWeb (11 punti)
- Esercizio di **programmazione** (11 punti)
- Esercizi di **testing** (6 + 5 = 11 punti)

La somma totale è **33 punti**, ma è necessario arrivare almeno a **5 punti su ciascuna delle tre parti** per passare l'esame.

Suggerimento: leggete **tutto** il testo prima di buttarvi a implementare "cose": la specifica dei test dell'esercizio 2 può aiutarvi a capire fin da subito cosa implementare (e cosa **NON** fare).

Per la consegna, caricate su AulaWeb la **solution "zippata"**, ovvero zippate la cartella che contiene il file `.sln` e i progetti, *dopo aver eseguito il Clean*. Codice che non compila o che non passa il `SanityCheck` (presente anche in versione `.txt` su AulaWeb) non verrà considerato. Dopo aver caricato la solution, per sicurezza, scaricate il vostro zip, scompattatelo in un'altra cartella e verificate che compili tutto.

Esercizio 1 (11 punti)

Implementare l'extension method `Slice<T>` che, dati:

- una sequenza (*potenzialmente infinita*) di elementi di tipo `T`, `source`
- un intero `size`

dovrà restituire una sequenza di *slice* ("fette"), ognuna rappresentata come un array di `T` (cioè, di tipo `T[]`), di dimensione `size`, a parte l'ultima "fetta" che potrà essere più piccola.

Se `size <= 0`, lanciare `ArgumentOutOfRangeException` (**immediatamente**, senza che sia necessario enumerare la sequenza restituita).

Enumerare la sequenza `source` **una sola volta**.

```
[Fact]
public void SanityCheck() {
    IEnumerable<int[]> x = Array.Empty<int>().Slice(2);
    _ = new List<string> { "a" }.Slice(1);
}
```

Esercizio 2 (6 punti)

Implementare, usando xUnit ed eventualmente Moq, i seguenti test relativi all'esercizio 1:

1. Casi "buoni"
    - array vuoto `[]`, con `size = 3` deve restituire la sequenza vuota
    - lista di interi `[1,2,3,4,5]` con `size = 2` deve restituire una sequenza con i tre slice `[1,2]`, `[3,4]`, `[5]` (in questo ordine)
    - lista di interi `[1,2,3]` con `size = 3` deve restituire una sequenza (di un singolo elemento) con la slice `[1,2,3]`
    - array di stringhe `["qui", "quo", "qua"]` con `size = 7` deve restituire una sequenza (di un singolo elemento) con la slice `["qui", "quo", "qua"]`
    - lista di stringhe `["qui", "quo", "qua"]` con `size = 1` deve restituire una sequenza con i tre slice `["qui"]`, `["quo"]`, `["qua"]`
2. Eccezioni
    - `size = -123` (con qualsiasi `source`) deve **immediatamente** sollevare `ArgumentOutOfRangeException`

Esercizio 3 (5 punti)

Verificare che l'implementazione non enumeri la sequenza `source` finché non diventa necessario e che la enumeri una sola volta.

Suggerimento: create una vostra sequenza che incrementa un contatore quando viene enumerata e poi verificate, in due test separati, che:

- dopo `var slices = source.Slice(2);` il contatore sia ancora `0`
- dopo aver enumerato (es. `slices.ToList()`) il contatore sia `1`